SQL Aggregation - Student Notes with Examples

What is Aggregation?

Aggregation in SQL combines multiple rows of data into a single summary result. It's like taking individual data points and calculating meaningful statistics from them.

Real-world analogy: Think of aggregation like a teacher calculating class statistics from individual student scores - finding the class average, counting how many students passed, or identifying the highest score.

Sample Tables for Examples

We'll use these two tables throughout our examples:

EMPLOYEES Table:

emp_id	name	department	salary	hire_date
1	John	Sales	50000	2023-01-15
2	Sarah	Sales	55000	2023-03-20
3	Mike	IT	60000	2022-08-10
4	Lisa	IT	65000	2023-02-05
5	Tom	HR	45000	2023-04-12
6	Anna	Sales	52000	2022-12-01

ORDERS Table:

order_id	customer_id	product	quantity	price	order_date
101	1	Laptop	2	1000	2024-01-15
102	2	Mouse	5	25	2024-01-16
103	1	Keyboard	3	50	2024-01-20
104	3	Laptop	1	1000	2024-02-01
105	2	Monitor	2	300	2024-02-05
106	1	Mouse	10	25	2024-02-10

Core Aggregate Functions

1. COUNT() - Counting Records

Purpose: Counts the number of rows or non-null values.

Syntax:

```
COUNT(*) -- Counts all rows
COUNT(column_name) -- Counts non-null values
```

Example 1: Basic Counting

```
-- Count total number of employees
SELECT COUNT(*) as total_employees FROM employees;
```

Result: `6`

Explanation: COUNT(*) counts every row in the table, regardless of NULL values.

Example 2: Counting by Department

```
-- Count employees in each department
SELECT department, COUNT(*) as employee_count
FROM employees
GROUP BY department;
```

Result:

```
department | employee_count
-----------|---------------
Sales      | 3
IT         | 2
HR         | 1
```

Explanation: GROUP BY groups rows with the same department value, then COUNT(*) counts rows in each group.

2. SUM() - Adding Values

Purpose: Calculates the total of numeric values.

Example 1: Total Payroll

```
-- Calculate total company payroll
SELECT SUM(salary) as total_payroll FROM employees;
```

Result: `327000`

Explanation: SUM adds all salary values: 50000 + 55000 + 60000 + 65000 + 45000 + 52000 = 327000

Example 2: Revenue by Customer

```
-- Calculate total revenue from each customer
SELECT customer_id, SUM(quantity * price) as total_spent
FROM orders
GROUP BY customer_id;
```

Result:

```
customer_id | total_spent
------------|------------
1           | 2400
2           | 725
3           | 1000
```

Explanation: For each customer, we multiply quantity × price for each order, then sum those totals. Customer 1: (2×1000) + (3×50) + (10×25) = 2000 + 150 + 250 = 2400

3. AVG() - Calculating Averages

Purpose: Calculates the arithmetic mean of numeric values.

Example 1: Average Salary

```
-- Find average salary across all employees
SELECT AVG(salary) as average_salary FROM employees;
```

Result: `54500`

Explanation: AVG adds all salaries and divides by count: 327000 ÷ 6 = 54500

Example 2: Average Salary by Department

```
-- Average salary in each department
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department;
```

Result:

```
department | avg_salary
-----------|----------
Sales      | 52333.33
```

IT | 62500
HR | 45000

Explanation: Sales department: (50000 + 55000 + 52000) ÷ 3 = 52333.33

4. MIN() and MAX() - Finding Extremes

Purpose: Find the smallest and largest values.

Example 1: Salary Range

```
-- Find lowest and highest salaries
SELECT MIN(salary) as lowest_salary, MAX(salary) as highest_salary
FROM employees;
```

Result:

```
lowest_salary | highest_salary
--------------|---------------
45000         | 65000
```

Explanation: MIN scans all salary values and returns the smallest (45000), MAX returns the largest (65000).

Example 2: Order Size Range by Product

```
-- Find minimum and maximum order quantities for each product
SELECT product, MIN(quantity) as min_order, MAX(quantity) as max_order
FROM orders
GROUP BY product;
```

Result:

```
product  | min_order | max_order
---------|-----------|----------
Laptop   | 1         | 2
Mouse    | 5         | 10
Keyboard | 3         | 3
Monitor  | 2         | 2
```

Explanation: For each product, MIN/MAX find the smallest and largest quantity ordered.

GROUP BY Clause

Purpose: Groups rows with the same values together so you can apply aggregate functions to each group.

Key Rule: Every column in SELECT (except aggregate functions) must appear in GROUP BY.

Example 1: Department Statistics

```
-- Get comprehensive statistics for each department
SELECT
    department,
    COUNT(*) as employee_count,
    AVG(salary) as avg_salary,
    MIN(salary) as min_salary,
    MAX(salary) as max_salary
FROM employees
GROUP BY department;
```

Result:

```
department | employee_count | avg_salary | min_salary | max_salary
-----------|----------------|------------|------------|----------
Sales      | 3              | 52333.33   | 50000      | 55000
IT         | 2              | 62500      | 60000      | 65000
HR         | 1              | 45000      | 45000      | 45000
```

Explanation: GROUP BY department creates three groups (Sales, IT, HR). Each aggregate function operates on the rows within each group separately.

Example 2: Monthly Order Analysis

```
-- Analyze orders by month
SELECT
    YEAR(order_date) as year,
    MONTH(order_date) as month,
    COUNT(*) as order_count,
    SUM(quantity * price) as total_revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;
```

Result:

```
year | month | order_count | total_revenue
```

-----|-------|-------------|-------------
2024 | 1 | 3 | 2175
2024 | 2 | 3 | 1850

Explanation: GROUP BY year and month creates groups for each month. January 2024 has 3 orders totaling $2,175 in revenue.

HAVING Clause

Purpose: Filters groups created by GROUP BY (like WHERE filters individual rows).

Key Difference:

- WHERE filters rows BEFORE grouping
- HAVING filters groups AFTER grouping

Example 1: Departments with High Average Salary

```
-- Find departments where average salary is above 50000
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Result:

```
department | avg_salary
-----------|----------
Sales      | 52333.33
IT         | 62500
```

Explanation: First, GROUP BY creates department groups. Then AVG(salary) is calculated for each group. Finally, HAVING filters out groups where the average is ≤ 50000 (HR department is excluded).

Example 2: Customers with Multiple Orders

```
-- Find customers who placed more than 2 orders
SELECT customer_id, COUNT(*) as order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 2;
```

Result:

```
customer_id | order_count
------------|------------
1           | 3
```

Explanation: GROUP BY customer_id groups orders by customer. COUNT(*) counts orders per customer. HAVING filters to show only customers with more than 2 orders.

Query Execution Order

Understanding the order SQL processes clauses helps avoid common mistakes:

```
SELECT department, COUNT(*) as emp_count    -- 5. Select and alias results
FROM employees                      -- 1. Get data from table
WHERE salary > 50000                  -- 2. Filter individual rows
GROUP BY department                    -- 3. Group filtered rows
HAVING COUNT(*) > 1                   -- 4. Filter groups
ORDER BY emp_count DESC;                 -- 6. Sort final results
```

Step-by-step execution:

1. **FROM:** Start with employees table
2. **WHERE:** Keep only employees with salary > 50000 (John, Sarah, Mike, Lisa, Anna)
3. **GROUP BY:** Group remaining employees by department (Sales: 3, IT: 2)
4. **HAVING:** Keep only groups with COUNT(*) > 1 (both Sales and IT qualify)
5. **SELECT:** Choose department and count for display
6. **ORDER BY:** Sort by employee count descending

SQL Filtering Data

1. Introduction to Filtering {#introduction}

Filtering in SQL means selecting only specific rows from a table that meet certain conditions. This is essential for:

- Finding specific records
- Analyzing subsets of data
- Improving query performance
- Creating reports with relevant data

The primary tool for filtering is the **WHERE clause**.

---

2. The WHERE Clause {#where-clause}

Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Basic Example

```
-- Sample table: employees
-- id | name     | age | salary | department
-- 1  | John     | 25  | 50000  | IT
-- 2  | Sarah    | 30  | 60000  | HR
-- 3  | Mike     | 35  | 70000  | IT
-- 4  | Lisa     | 28  | 55000  | Finance

SELECT name, salary
FROM employees
WHERE department <> 'HR';
```

Result:

```
name  | salary
------|-------
John  | 50000
Mike  | 70000
```

---

3. Comparison Operators {#comparison-operators}

Operator	Description	Example
`=`	Equal to	`age = 25`
`<>` or `!=`	Not equal to	`department <> 'HR'`
`<`	Less than	`salary < 60000`
`>`	Greater than	`age > 30`
`<=`	Less than or equal	`age <= 30`
`>=`	Greater than or equal	`salary >= 55000`

Examples

```
-- Find employees older than 30
SELECT name, age
FROM employees
WHERE age > 30;

-- Find employees with salary not equal to 60000
SELECT name, salary
FROM employees
WHERE salary != 60000;

-- Find employees with salary less than or equal to 55000
SELECT name, salary
FROM employees
WHERE salary <= 55000;
```

---

4. Logical Operators {#logical-operators}

AND Operator

Both conditions must be true.

```
SELECT name, age, salary
FROM employees
```

WHERE age > 25 AND salary > 55000;

Result:

name	age	salary
Sarah	30	60000
Mike	35	70000

OR Operator

At least one condition must be true.

SELECT name, department
FROM employees
WHERE department = 'IT' OR department = 'HR';

NOT Operator

Negates a condition.

SELECT name, department
FROM employees
WHERE NOT department = 'IT';

AND OPERATOR

A	B	RESULT
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
TRUE	TRUE	TRUE
FALSE	FALSE	FALSE

OR OPERATOR

A	B	RESULT
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
TRUE	TRUE	TRUE

FALSE **FALSE** **FALSE**

Combining Logical Operators

Use parentheses to control order of evaluation.

```
SELECT name, age, salary, department
FROM employees
WHERE (age > 25 AND salary > 55000) OR department = 'HR';
```

---

5. Pattern Matching {#pattern-matching}

LIKE Operator

Used for pattern matching with wildcards.

Wildcard	Description	Example
`%`	Zero or more characters	`name LIKE 'J%'`
`_`	Exactly one character	`name LIKE 'J_hn'`

Examples

```
-- Sample table: products
-- id | product_name    | category | price
-- 1  | iPhone 14       | Phone    | 999
-- 2  | iPad Pro        | Tablet   | 1099
-- 3  | MacBook Air     | Laptop   | 1199
-- 4  | Samsung Galaxy  | Phone    | 899

-- Find products starting with 'i'
SELECT product_name, price
FROM products
WHERE product_name LIKE 'i%';

-- Find products ending with 'Pro'
SELECT product_name, category
FROM products
WHERE product_name LIKE '%Pro';
```

```
-- Find products with exactly 4 characters in the name
SELECT product_name
FROM products
WHERE product_name LIKE '____';
```

Case-Sensitive Matching

```
-- Case-insensitive (most databases)
SELECT * FROM products WHERE product_name LIKE 'iphone%';

-- Case-sensitive (some databases)
SELECT * FROM products WHERE product_name LIKE BINARY 'iPhone%';
```

7. Filtering with Ranges {#ranges}

BETWEEN Operator

Selects values within a range (inclusive).

```
-- Find employees with salaries between 50000 and 60000
SELECT name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 60000;

-- Find employees aged between 25 and 30
SELECT name, age
FROM employees
WHERE age BETWEEN 25 AND 30;
```

Date Ranges

```
-- Sample table: orders
-- order_id | customer_name | order_date  | amount
-- 1        | John Smith    | 2023-01-15  | 250.00
-- 2        | Jane Doe      | 2023-02-20  | 180.50

SELECT order_id, customer_name, amount
FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-01-31';
```

---

8. Filtering with Lists {#lists}

IN Operator

Checks if value matches any value in a list.

```
-- Find employees in specific departments
SELECT name, department
FROM employees
WHERE department IN ('IT', 'HR', 'Finance');

-- Find products in specific price ranges
SELECT product_name, price
FROM products
WHERE price IN (999, 1099, 1199);
```

NOT IN Operator

```
-- Find employees NOT in IT or HR
SELECT name, department
FROM employees
WHERE department NOT IN ('IT', 'HR');
```

SQL Data Manipulation Language (DML) - Student Notes

What is DML?

DML (Data Manipulation Language) is used to manipulate data stored in database tables. The three main DML commands are:

- **INSERT** - Add new records to a table
- **UPDATE** - Modify existing records in a table
- **DELETE** - Remove records from a table

---

1. INSERT Statement

The INSERT statement adds new rows of data to a table.

Syntax

```
-- Insert all columns
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);

-- Insert without specifying columns (must provide all values in order)
INSERT INTO table_name
VALUES (value1, value2, value3, ...);

-- Insert multiple rows at once
INSERT INTO table_name (column1, column2, column3)
VALUES
    (value1, value2, value3),
    (value4, value5, value6),
    (value7, value8, value9);
```

Example Database Setup

```
-- Create a students table
    age INT,CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
```

```
    major VARCHAR(50),
    gpa DECIMAL(3,2),
    enrollment_date DATE
);
```

INSERT Examples

Example 1: Insert a single record (all columns)

```
INSERT INTO students (student_id, first_name, last_name, email, age, major, gpa,
enrollment_date)
VALUES (1, 'John', 'Doe', 'john.doe@email.com', 20, 'Computer Science', 3.75,
'2024-09-01');
```

Example 2: Insert without specifying all columns

```
-- student_id will auto-increment, other columns can be NULL
INSERT INTO students (first_name, last_name, email, major)
VALUES ('Jane', 'Smith', 'jane.smith@email.com', 'Mathematics');
```

Example 3: Insert multiple rows at once

```
INSERT INTO students (first_name, last_name, email, age, major, gpa, enrollment_date)
VALUES
    ('Bob', 'Johnson', 'bob.j@email.com', 21, 'Physics', 3.45, '2024-09-01'),
    ('Alice', 'Williams', 'alice.w@email.com', 19, 'Chemistry', 3.90, '2024-09-01'),
    ('Charlie', 'Brown', 'charlie.b@email.com', 22, 'Biology', 3.20, '2024-09-01');
```

Example 4: Insert with NULL values

```
INSERT INTO students (first_name, last_name, age, major)
VALUES ('David', 'Lee', 20, 'Engineering');
-- email, gpa, and enrollment_date will be NULL
```

Example 5: Insert all values without column names

```
INSERT INTO students
VALUES (6, 'Emma', 'Davis', 'emma.d@email.com', 21, 'History', 3.65, '2024-09-01');
-- Must provide values for ALL columns in exact order
```

---

2. UPDATE Statement

The UPDATE statement modifies existing records in a table.

Syntax

```
-- Update specific columns
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;

-- Update without WHERE (updates ALL rows - USE WITH CAUTION!)
UPDATE table_name
SET column1 = value1;
```

⚠️ Important Warning

Always use a WHERE clause with UPDATE! Without it, ALL rows in the table will be updated.

UPDATE Examples

Example 6: Update a single column for one student

```
UPDATE students
SET gpa = 3.85
WHERE student_id = 1;
```

Example 7: Update multiple columns for one student

```
UPDATE students
SET email = 'john.doe.new@email.com', major = 'Data Science', gpa = 3.80
WHERE student_id = 1;
```

Example 8: Update based on a condition

```
-- Give all Computer Science students a 0.1 GPA boost
UPDATE students
SET gpa = gpa + 0.1
WHERE major = 'Computer Science';
```

Example 9: Update multiple students with same major

```
UPDATE students
SET enrollment_date = '2024-09-15'
WHERE major = 'Mathematics';
```

Example 10: Update using multiple conditions

```
-- Update students who are 20 or older and have GPA below 3.0
UPDATE students
SET gpa = 3.0
```

```
WHERE age >= 20 AND gpa < 3.0;
```

Example 11: Update with NULL value

```
-- Clear the email for a specific student
UPDATE students
SET email = NULL
WHERE student_id = 3;
```

Example 12: Update using CASE statement (conditional update)

```
UPDATE students
SET gpa = CASE
    WHEN gpa < 2.0 THEN 2.0
    WHEN gpa > 4.0 THEN 4.0
    ELSE gpa
END;
-- This ensures all GPAs are between 2.0 and 4.0
```

---

3. DELETE Statement

The DELETE statement removes records from a table.

Syntax

```
-- Delete specific rows
DELETE FROM table_name
WHERE condition;

-- Delete ALL rows (USE WITH EXTREME CAUTION!)
DELETE FROM table_name;

-- Truncate (faster way to delete all rows)
TRUNCATE TABLE table_name;
```

⚠️ Important Warning

Always use a WHERE clause with DELETE! Without it, ALL rows in the table will be deleted.

DELETE Examples

Example 13: Delete a single student by ID

```
DELETE FROM students
WHERE student_id = 5;
```

Example 14: Delete students based on a condition

```
-- Delete all students with GPA below 2.0
DELETE FROM students
WHERE gpa < 2.0;
```

Example 15: Delete using multiple conditions

```
-- Delete students who are Chemistry majors AND have no email
DELETE FROM students
WHERE major = 'Chemistry' AND email IS NULL;
```

Example 16: Delete students by age range

```
DELETE FROM students
WHERE age < 18 OR age > 25;
```

Example 17: Delete using NOT IN

```
-- Delete students NOT majoring in specific subjects
DELETE FROM students
WHERE major NOT IN ('Computer Science', 'Mathematics', 'Physics');
```

Complete Practice Script

```
-- ============================================
-- SETUP: Create and populate the table
-- ============================================

CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    age INT,
    major VARCHAR(50),
    gpa DECIMAL(3,2),
    enrollment_date DATE
);

-- ============================================
-- INSERT EXAMPLES
-- ============================================

-- Single insert with all columns
```

```
INSERT INTO students (student_id, first_name, last_name, email, age, major, gpa,
enrollment_date)
VALUES (1, 'John', 'Doe', 'john.doe@email.com', 20, 'Computer Science', 3.75,
'2024-09-01');

-- Insert without some columns
INSERT INTO students (first_name, last_name, email, major)
VALUES ('Jane', 'Smith', 'jane.smith@email.com', 'Mathematics');

-- Multiple inserts
INSERT INTO students (first_name, last_name, email, age, major, gpa, enrollment_date)
VALUES
    ('Bob', 'Johnson', 'bob.j@email.com', 21, 'Physics', 3.45, '2024-09-01'),
    ('Alice', 'Williams', 'alice.w@email.com', 19, 'Chemistry', 3.90, '2024-09-01'),
    ('Charlie', 'Brown', 'charlie.b@email.com', 22, 'Biology', 3.20, '2024-09-01'),
    ('David', 'Lee', 'david.lee@email.com', 20, 'Engineering', 2.95, '2024-09-01'),
    ('Emma', 'Davis', 'emma.d@email.com', 21, 'History', 3.65, '2024-09-01'),
    ('Frank', 'Miller', NULL, 23, 'Computer Science', 2.50, '2024-09-01'),
    ('Grace', 'Wilson', 'grace.w@email.com', 19, 'Mathematics', 3.85, '2024-09-01'),
    ('Henry', 'Moore', 'henry.m@email.com', 24, 'Physics', 1.95, '2024-09-01');

-- View all students
SELECT * FROM students;

-- ============================================
-- UPDATE EXAMPLES
-- ============================================

-- Update single column
UPDATE students
SET gpa = 3.85
WHERE student_id = 1;

-- Update multiple columns
UPDATE students
SET email = 'jane.smith.new@email.com', gpa = 3.70
WHERE first_name = 'Jane' AND last_name = 'Smith';

-- Update with calculation
UPDATE students
SET gpa = gpa + 0.1
WHERE major = 'Computer Science' AND gpa < 3.5;

-- Update with condition
UPDATE students
SET gpa = 2.0
WHERE gpa < 2.0;
```

```sql
-- Update using CASE
UPDATE students
SET major = CASE
    WHEN major = 'Computer Science' THEN 'CS'
    WHEN major = 'Mathematics' THEN 'Math'
    ELSE major
END;

-- Update with NULL
UPDATE students
SET email = NULL
WHERE student_id = 8;

-- View updated data
SELECT * FROM students;

-- ============================================
-- DELETE EXAMPLES
-- ============================================

-- Delete by ID
DELETE FROM students
WHERE student_id = 10;

-- Delete by condition
DELETE FROM students
WHERE gpa < 2.0;

-- Delete with multiple conditions
DELETE FROM students
WHERE major = 'History' AND age > 22;

-- Delete using NULL check
DELETE FROM students
WHERE email IS NULL;

-- Delete using NOT IN
DELETE FROM students
WHERE major NOT IN ('CS', 'Math', 'Physics');

-- View remaining data
SELECT * FROM students;

-- ============================================
-- SAFETY CHECKS BEFORE DELETE/UPDATE
-- ============================================

-- Always SELECT first to see what will be affected
```

```
SELECT * FROM students WHERE gpa < 2.5;
-- If the results look correct, then run the DELETE
-- DELETE FROM students WHERE gpa < 2.5;

-- Count affected rows before update
SELECT COUNT(*) FROM students WHERE major = 'Physics';
-- If the count is correct, then run the UPDATE
-- UPDATE students SET major = 'Applied Physics' WHERE major = 'Physics';
```

Important Differences Between Commands

Command	Purpose	Affects	Can Use WHERE
INSERT	Add new rows	New data only	No (not applicable)
UPDATE	Modify existing rows	Existing data	Yes (recommended)
DELETE	Remove rows	Existing data	Yes (recommended)

Safety Best Practices

1. Always Use WHERE Clause

```
-- ❌ DANGEROUS: Updates ALL rows
UPDATE students SET gpa = 4.0;

-- ✅ SAFE: Updates specific rows
UPDATE students SET gpa = 4.0 WHERE student_id = 1;
```

2. SELECT Before DELETE/UPDATE

```
-- Step 1: Check what will be affected
SELECT * FROM students WHERE gpa < 2.0;

-- Step 2: If results look correct, run the DELETE
DELETE FROM students WHERE gpa < 2.0;
```

3. Use Transactions (Advanced)

```
-- Start transaction
START TRANSACTION;
```

```
-- Make changes
UPDATE students SET gpa = gpa + 0.5 WHERE major = 'Physics';

-- Check the changes
SELECT * FROM students WHERE major = 'Physics';

-- If correct, commit; if wrong, rollback
COMMIT;
-- or
ROLLBACK;
```

4. Backup Before Major Changes

```
-- Create a backup table
CREATE TABLE students_backup AS SELECT * FROM students;

-- Now you can safely make changes
UPDATE students SET gpa = gpa * 1.1;

-- If something goes wrong, restore from backup
-- DROP TABLE students;
-- RENAME TABLE students_backup TO students;
```

Real-World Scenarios

Scenario 1: Student Registration System

```
-- Register a new student
INSERT INTO students (first_name, last_name, email, age, major, enrollment_date)
VALUES ('Michael', 'Scott', 'michael.scott@email.com', 21, 'Business', CURDATE());

-- Update student's major
UPDATE students
SET major = 'Marketing'
WHERE email = 'michael.scott@email.com';

-- Student withdraws
DELETE FROM students
WHERE email = 'michael.scott@email.com';
```

Scenario 2: Bulk Grade Update

```
-- Add bonus points to all students in Computer Science
UPDATE students
SET gpa = LEAST(gpa + 0.2, 4.0) -- Cap at 4.0
```

```
WHERE major = 'Computer Science';
```

Scenario 3: Data Cleanup

```
-- Remove students with no email after 30 days
DELETE FROM students
WHERE email IS NULL
  AND enrollment_date < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

SQL Data Manipulation Language (DML) - Student Notes

What is DML?

DML (Data Manipulation Language) is used to manipulate data stored in database tables. The three main DML commands are:

- **INSERT** - Add new records to a table
- **UPDATE** - Modify existing records in a table
- **DELETE** - Remove records from a table

---

1. INSERT Statement

The INSERT statement adds new rows of data to a table.

Syntax

```
-- Insert all columns
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);

-- Insert without specifying columns (must provide all values in order)
INSERT INTO table_name
VALUES (value1, value2, value3, ...);

-- Insert multiple rows at once
INSERT INTO table_name (column1, column2, column3)
VALUES
    (value1, value2, value3),
    (value4, value5, value6),
    (value7, value8, value9);
```

Example Database Setup

```
-- Create a students table
    age INT,CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
```

```
    major VARCHAR(50),
    gpa DECIMAL(3,2),
    enrollment_date DATE
);
```

INSERT Examples

Example 1: Insert a single record (all columns)

```
INSERT INTO students (student_id, first_name, last_name, email, age, major, gpa,
enrollment_date)
VALUES (1, 'John', 'Doe', 'john.doe@email.com', 20, 'Computer Science', 3.75,
'2024-09-01');
```

Example 2: Insert without specifying all columns

```
-- student_id will auto-increment, other columns can be NULL
INSERT INTO students (first_name, last_name, email, major)
VALUES ('Jane', 'Smith', 'jane.smith@email.com', 'Mathematics');
```

Example 3: Insert multiple rows at once

```
INSERT INTO students (first_name, last_name, email, age, major, gpa, enrollment_date)
VALUES
    ('Bob', 'Johnson', 'bob.j@email.com', 21, 'Physics', 3.45, '2024-09-01'),
    ('Alice', 'Williams', 'alice.w@email.com', 19, 'Chemistry', 3.90, '2024-09-01'),
    ('Charlie', 'Brown', 'charlie.b@email.com', 22, 'Biology', 3.20, '2024-09-01');
```

Example 4: Insert with NULL values

```
INSERT INTO students (first_name, last_name, age, major)
VALUES ('David', 'Lee', 20, 'Engineering');
-- email, gpa, and enrollment_date will be NULL
```

Example 5: Insert all values without column names

```
INSERT INTO students
VALUES (6, 'Emma', 'Davis', 'emma.d@email.com', 21, 'History', 3.65, '2024-09-01');
-- Must provide values for ALL columns in exact order
```

---

2. UPDATE Statement

The UPDATE statement modifies existing records in a table.

Syntax

```
-- Update specific columns
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;

-- Update without WHERE (updates ALL rows - USE WITH CAUTION!)
UPDATE table_name
SET column1 = value1;
```

⚠️ Important Warning

Always use a WHERE clause with UPDATE! Without it, ALL rows in the table will be updated.

UPDATE Examples

Example 6: Update a single column for one student

```
UPDATE students
SET gpa = 3.85
WHERE student_id = 1;
```

Example 7: Update multiple columns for one student

```
UPDATE students
SET email = 'john.doe.new@email.com', major = 'Data Science', gpa = 3.80
WHERE student_id = 1;
```

Example 8: Update based on a condition

```
-- Give all Computer Science students a 0.1 GPA boost
UPDATE students
SET gpa = gpa + 0.1
WHERE major = 'Computer Science';
```

Example 9: Update multiple students with same major

```
UPDATE students
SET enrollment_date = '2024-09-15'
WHERE major = 'Mathematics';
```

Example 10: Update using multiple conditions

```
-- Update students who are 20 or older and have GPA below 3.0
UPDATE students
SET gpa = 3.0
```

```
WHERE age >= 20 AND gpa < 3.0;
```

Example 11: Update with NULL value

```
-- Clear the email for a specific student
UPDATE students
SET email = NULL
WHERE student_id = 3;
```

Example 12: Update using CASE statement (conditional update)

```
UPDATE students
SET gpa = CASE
    WHEN gpa < 2.0 THEN 2.0
    WHEN gpa > 4.0 THEN 4.0
    ELSE gpa
END;
-- This ensures all GPAs are between 2.0 and 4.0
```

---

3. DELETE Statement

The DELETE statement removes records from a table.

Syntax

```
-- Delete specific rows
DELETE FROM table_name
WHERE condition;

-- Delete ALL rows (USE WITH EXTREME CAUTION!)
DELETE FROM table_name;

-- Truncate (faster way to delete all rows)
TRUNCATE TABLE table_name;
```

⚠️ Important Warning

Always use a WHERE clause with DELETE! Without it, ALL rows in the table will be deleted.

DELETE Examples

Example 13: Delete a single student by ID

```
DELETE FROM students
WHERE student_id = 5;
```

Example 14: Delete students based on a condition

```
-- Delete all students with GPA below 2.0
DELETE FROM students
WHERE gpa < 2.0;
```

Example 15: Delete using multiple conditions

```
-- Delete students who are Chemistry majors AND have no email
DELETE FROM students
WHERE major = 'Chemistry' AND email IS NULL;
```

Example 16: Delete students by age range

```
DELETE FROM students
WHERE age < 18 OR age > 25;
```

Example 17: Delete using NOT IN

```
-- Delete students NOT majoring in specific subjects
DELETE FROM students
WHERE major NOT IN ('Computer Science', 'Mathematics', 'Physics');
```

Complete Practice Script

```
-- ============================================
-- SETUP: Create and populate the table
-- ============================================

CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100),
    age INT,
    major VARCHAR(50),
    gpa DECIMAL(3,2),
    enrollment_date DATE
);

-- ============================================
-- INSERT EXAMPLES
-- ============================================

-- Single insert with all columns
```

```
INSERT INTO students (student_id, first_name, last_name, email, age, major, gpa,
enrollment_date)
VALUES (1, 'John', 'Doe', 'john.doe@email.com', 20, 'Computer Science', 3.75,
'2024-09-01');

-- Insert without some columns
INSERT INTO students (first_name, last_name, email, major)
VALUES ('Jane', 'Smith', 'jane.smith@email.com', 'Mathematics');

-- Multiple inserts
INSERT INTO students (first_name, last_name, email, age, major, gpa, enrollment_date)
VALUES
    ('Bob', 'Johnson', 'bob.j@email.com', 21, 'Physics', 3.45, '2024-09-01'),
    ('Alice', 'Williams', 'alice.w@email.com', 19, 'Chemistry', 3.90, '2024-09-01'),
    ('Charlie', 'Brown', 'charlie.b@email.com', 22, 'Biology', 3.20, '2024-09-01'),
    ('David', 'Lee', 'david.lee@email.com', 20, 'Engineering', 2.95, '2024-09-01'),
    ('Emma', 'Davis', 'emma.d@email.com', 21, 'History', 3.65, '2024-09-01'),
    ('Frank', 'Miller', NULL, 23, 'Computer Science', 2.50, '2024-09-01'),
    ('Grace', 'Wilson', 'grace.w@email.com', 19, 'Mathematics', 3.85, '2024-09-01'),
    ('Henry', 'Moore', 'henry.m@email.com', 24, 'Physics', 1.95, '2024-09-01');

-- View all students
SELECT * FROM students;

-- ============================================
-- UPDATE EXAMPLES
-- ============================================

-- Update single column
UPDATE students
SET gpa = 3.85
WHERE student_id = 1;

-- Update multiple columns
UPDATE students
SET email = 'jane.smith.new@email.com', gpa = 3.70
WHERE first_name = 'Jane' AND last_name = 'Smith';

-- Update with calculation
UPDATE students
SET gpa = gpa + 0.1
WHERE major = 'Computer Science' AND gpa < 3.5;

-- Update with condition
UPDATE students
SET gpa = 2.0
WHERE gpa < 2.0;
```

```sql
-- Update using CASE
UPDATE students
SET major = CASE
    WHEN major = 'Computer Science' THEN 'CS'
    WHEN major = 'Mathematics' THEN 'Math'
    ELSE major
END;

-- Update with NULL
UPDATE students
SET email = NULL
WHERE student_id = 8;

-- View updated data
SELECT * FROM students;

-- ============================================
-- DELETE EXAMPLES
-- ============================================

-- Delete by ID
DELETE FROM students
WHERE student_id = 10;

-- Delete by condition
DELETE FROM students
WHERE gpa < 2.0;

-- Delete with multiple conditions
DELETE FROM students
WHERE major = 'History' AND age > 22;

-- Delete using NULL check
DELETE FROM students
WHERE email IS NULL;

-- Delete using NOT IN
DELETE FROM students
WHERE major NOT IN ('CS', 'Math', 'Physics');

-- View remaining data
SELECT * FROM students;

-- ============================================
-- SAFETY CHECKS BEFORE DELETE/UPDATE
-- ============================================

-- Always SELECT first to see what will be affected
```

```
SELECT * FROM students WHERE gpa < 2.5;
-- If the results look correct, then run the DELETE
-- DELETE FROM students WHERE gpa < 2.5;

-- Count affected rows before update
SELECT COUNT(*) FROM students WHERE major = 'Physics';
-- If the count is correct, then run the UPDATE
-- UPDATE students SET major = 'Applied Physics' WHERE major = 'Physics';
```

Important Differences Between Commands

Command	Purpose	Affects	Can Use WHERE
INSERT	Add new rows	New data only	No (not applicable)
UPDATE	Modify existing rows	Existing data	Yes (recommended)
DELETE	Remove rows	Existing data	Yes (recommended)

Safety Best Practices

1. Always Use WHERE Clause

```
-- ❌ DANGEROUS: Updates ALL rows
UPDATE students SET gpa = 4.0;

-- ✅ SAFE: Updates specific rows
UPDATE students SET gpa = 4.0 WHERE student_id = 1;
```

2. SELECT Before DELETE/UPDATE

```
-- Step 1: Check what will be affected
SELECT * FROM students WHERE gpa < 2.0;

-- Step 2: If results look correct, run the DELETE
DELETE FROM students WHERE gpa < 2.0;
```

3. Use Transactions (Advanced)

```
-- Start transaction
START TRANSACTION;
```

```
-- Make changes
UPDATE students SET gpa = gpa + 0.5 WHERE major = 'Physics';

-- Check the changes
SELECT * FROM students WHERE major = 'Physics';

-- If correct, commit; if wrong, rollback
COMMIT;
-- or
ROLLBACK;
```

4. Backup Before Major Changes

```
-- Create a backup table
CREATE TABLE students_backup AS SELECT * FROM students;

-- Now you can safely make changes
UPDATE students SET gpa = gpa * 1.1;

-- If something goes wrong, restore from backup
-- DROP TABLE students;
-- RENAME TABLE students_backup TO students;
```

Real-World Scenarios

Scenario 1: Student Registration System

```
-- Register a new student
INSERT INTO students (first_name, last_name, email, age, major, enrollment_date)
VALUES ('Michael', 'Scott', 'michael.scott@email.com', 21, 'Business', CURDATE());

-- Update student's major
UPDATE students
SET major = 'Marketing'
WHERE email = 'michael.scott@email.com';

-- Student withdraws
DELETE FROM students
WHERE email = 'michael.scott@email.com';
```

Scenario 2: Bulk Grade Update

```
-- Add bonus points to all students in Computer Science
UPDATE students
SET gpa = LEAST(gpa + 0.2, 4.0) -- Cap at 4.0
```

```
WHERE major = 'Computer Science';
```

Scenario 3: Data Cleanup

```
-- Remove students with no email after 30 days
DELETE FROM students
WHERE email IS NULL
  AND enrollment_date < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

SQL Joins - Comprehensive Guide

What are Joins?

Joins are used to combine rows from two or more tables based on a related column between them. They allow you to retrieve data that is spread across multiple tables in a relational database.

Types of Joins

1. INNER JOIN

Returns only the rows where there is a match in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Example:

```
-- Tables: Employees and Departments
SELECT Employees.name, Employees.salary, Departments.dept_name
FROM Employees
INNER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

Result: Only employees who have a matching department will be displayed.

---

2. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table and matching rows from the right table. If no match exists, NULL values are returned for right table columns.

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
```

ON table1.column = table2.column;

Example:

```
SELECT Employees.name, Departments.dept_name
FROM Employees
LEFT JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

Result: All employees are shown, even if they don't have a department assigned (dept_name will be NULL for those).

---

3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table. If no match exists, NULL values are returned for left table columns.

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Example:

```
SELECT Employees.name, Departments.dept_name
FROM Employees
RIGHT JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

Result: All departments are shown, even if no employees work in them (employee name will be NULL for empty departments).

---

4. FULL OUTER JOIN

Returns all rows when there is a match in either left or right table. Returns NULL for non-matching rows from both sides.

Syntax:

```
SELECT columns
```

```
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column;
```

Example:

```
SELECT Employees.name, Departments.dept_name
FROM Employees
FULL OUTER JOIN Departments
ON Employees.dept_id = Departments.dept_id;
```

Result: All employees and all departments, with NULLs where there's no match on either side.

Note: MySQL doesn't support FULL OUTER JOIN directly. You can achieve it using UNION of LEFT and RIGHT joins.

---

5. CROSS JOIN

Returns the Cartesian product of both tables (every row from table1 combined with every row from table2).

Syntax:

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Example:

```
SELECT Employees.name, Departments.dept_name
FROM Employees
CROSS JOIN Departments;
```

Result: If you have 5 employees and 3 departments, you get 15 rows (5 × 3).

---

6. SELF JOIN

A table is joined with itself to compare rows within the same table.

Syntax:

```
SELECT columns
FROM table1 t1
JOIN table1 t2
ON condition;
```

Example:

```
-- Find employees and their managers (both in the same Employees table)
SELECT e1.name AS Employee, e2.name AS Manager
FROM Employees e1
INNER JOIN Employees e2
ON e1.manager_id = e2.employee_id;
```

Practical Examples with Sample Data

Sample Tables:

Employees Table:

emp_id	name	dept_id	salary
1	John	101	50000
2	Sarah	102	60000
3	Mike	NULL	55000
4	Lisa	101	52000

Departments Table:

dept_id	dept_name
101	Sales
102	Marketing
103	HR

Example Queries:

1. INNER JOIN - Employees with departments:

```
SELECT e.name, d.dept_name, e.salary
FROM Employees e
INNER JOIN Departments d
ON e.dept_id = d.dept_id;
```

Result: John, Sarah, Lisa (Mike excluded because dept_id is NULL)

2. LEFT JOIN - All employees:

```
SELECT e.name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_id = d.dept_id;
```

Result: All 4 employees, Mike shows NULL for dept_name

3. RIGHT JOIN - All departments:

```
SELECT e.name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_id = d.dept_id;
```

Result: HR department shows with NULL employee name, Mike not included

---

Multiple Joins

You can join more than two tables:

```
SELECT e.name, d.dept_name, p.project_name
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.dept_id
INNER JOIN Projects p ON e.emp_id = p.emp_id;
```

Join with WHERE and Other Clauses

```
SELECT e.name, d.dept_name, e.salary
FROM Employees e
INNER JOIN Departments d
ON e.dept_id = d.dept_id
WHERE e.salary > 50000
ORDER BY e.salary DESC;
```

Advanced Join Techniques

Join with Aggregate Functions

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count, AVG(e.salary) AS
avg_salary
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Join with Subqueries

```
SELECT e.name, e.salary, dept_avg.avg_sal
FROM Employees e
INNER JOIN (
   SELECT dept_id, AVG(salary) AS avg_sal
   FROM Employees
   GROUP BY dept_id
) dept_avg ON e.dept_id = dept_avg.dept_id
WHERE e.salary > dept_avg.avg_sal;
```

Natural Join

Automatically joins tables based on columns with the same name:

```
SELECT name, dept_name
FROM Employees
NATURAL JOIN Departments;
```

Warning: Use with caution as it can produce unexpected results if column names match unintentionally.

Key Points to Remember

- **INNER JOIN** is the most common and returns only matches
- **LEFT/RIGHT JOIN** preserve all rows from one table
- **FULL OUTER JOIN** preserves all rows from both tables
- Always use **table aliases** (e.g., e, d) for readability with joins
- Join conditions typically use **foreign key relationships**
- You can join on multiple conditions using **AND** in the ON clause

- **Performance tip:** Index the columns used in JOIN conditions
- The order of tables in LEFT/RIGHT JOIN matters significantly
- INNER JOIN is commutative (order doesn't matter for results)

Common Interview Questions

Q1: What's the difference between WHERE and ON in joins?

- **ON** specifies the join condition
- **WHERE** filters the result after joining
- For INNER JOIN, they work similarly, but for OUTER JOINs, the difference is crucial

```
-- These give different results with LEFT JOIN:

-- Filters before joining (keeps all left rows):
LEFT JOIN table2 ON table1.id = table2.id AND table2.status = 'active'

-- Filters after joining (removes left rows):
LEFT JOIN table2 ON table1.id = table2.id WHERE table2.status = 'active'
```

Q2: What's the difference between INNER JOIN and WHERE with comma-separated tables?

```
-- Old style (implicit join)
SELECT * FROM table1, table2 WHERE table1.id = table2.id;

-- Modern style (explicit join) - PREFERRED
SELECT * FROM table1 INNER JOIN table2 ON table1.id = table2.id;
```

Both produce the same result, but explicit JOIN syntax is clearer and recommended.

Q3: How do you find records that don't have a match?

```
-- Find employees without departments
SELECT e.name
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Performance Optimization Tips

1. **Index join columns** - Create indexes on columns used in ON clauses

2. **Use appropriate join types** - Don't use LEFT JOIN if INNER JOIN suffices
3. **Filter early** - Use WHERE conditions to reduce dataset before joining when possible
4. **Avoid SELECT *** - Only select columns you need
5. **Analyze query execution plans** - Use EXPLAIN to understand how queries are executed
6. **Consider join order** - In some databases, the order of joins can affect performance

---

Common Pitfalls to Avoid

1. **Forgetting NULL values** - NULLs don't match in join conditions
2. **Cartesian product** - Forgetting ON clause creates CROSS JOIN
3. **Duplicate rows** - Many-to-many relationships can create duplicates
4. **Wrong join type** - Using INNER JOIN when you need LEFT JOIN loses data
5. **Ambiguous column names** - Always use table aliases to avoid confusion

---

Practice Exercises

Exercise 1: Basic Joins

Write a query to find all employees with their department names and only show employees earning more than 51000.

Exercise 2: Self Join

Create a query to find all pairs of employees who work in the same department.

Exercise 3: Multiple Joins

Join Employees, Departments, and Projects tables to show employee names, their departments, and projects they're working on.

Exercise 4: Aggregate with Join

Find the department with the highest average salary.

Exercise 5: NULL Handling

Find all departments that have no employees assigned to them.

Quick Reference Table

Join Type	Returns	Use Case
INNER JOIN	Only matching rows from both tables	Most common, when you need related data
LEFT JOIN	All rows from left + matching from right	Keep all records from main table
RIGHT JOIN	All rows from right + matching from left	Less common, opposite of LEFT JOIN
FULL OUTER JOIN	All rows from both tables	Rare, when you need everything
CROSS JOIN	Cartesian product	Combinations, very rare in practice
SELF JOIN	Table joined with itself	Hierarchical data, comparisons